Image-Native Language Models: Reading and Writing as Visual Generation

Imagized Language Model Project

June 2026

Abstract

Most large language models represent language through symbolic text tokens and treat images of writing as an auxiliary input modality. This paper sketches a different research direction: an image-native language model whose inputs and outputs are visual canvases. The model learns from printed pages, handwritten forms, oracle-bone glyphs, bronze inscriptions, seal forms, and other historical scripts as images. It answers by generating readable images containing modern language and historically grounded glyph panels. We propose a 3090-friendly staged architecture based on visual autoencoding, masked visual language modeling, latent diffusion, and retrieval-augmented glyph evolution.

1 Motivation

Human readers do not see byte-pair tokens. They see pages, lines, strokes, spacing, damage, font variation, and script-specific visual forms. Current language models are extremely capable, but their central representation is normally a sequence of text tokens. Multimodal models accept images, but many still produce text as the primary output interface. For example, GPT-4 is reported as accepting image and text inputs and producing text outputs; GPT-4o's API documentation lists text and image input with text output for that model. Llama 3 similarly remains a token-based dense Transformer family, with multimodal capabilities added compositionally.

The Imagized Language Model (ILM) goal is different:

$$\text{image input} \rightarrow \text{visual language state} \rightarrow \text{image output.}$$

The output should itself be a readable answer image. For a query such as "explain the evolution of character zhong", the system should produce a page containing modern explanatory writing plus oracle, bronze, seal, and modern forms.

2 Prior Art and Gap

OCR-free document models such as Donut and screenshot parsing models such as Pix2Struct show that document images can be understood without an explicit OCR pipeline. Token-free language models such as CANINE, ByT5, and MEGABYTE show that subword tokenization is not a hard requirement for language modeling. Discrete diffusion language models, including D3PM, Diffusion-LM, SEDD, and LLaDA-style masked diffusion, show that generation need not be left-to-right. Image generative models such as latent diffusion, MaskGIT, and DiT show that high-quality image generation can be performed in compressed latent spaces.

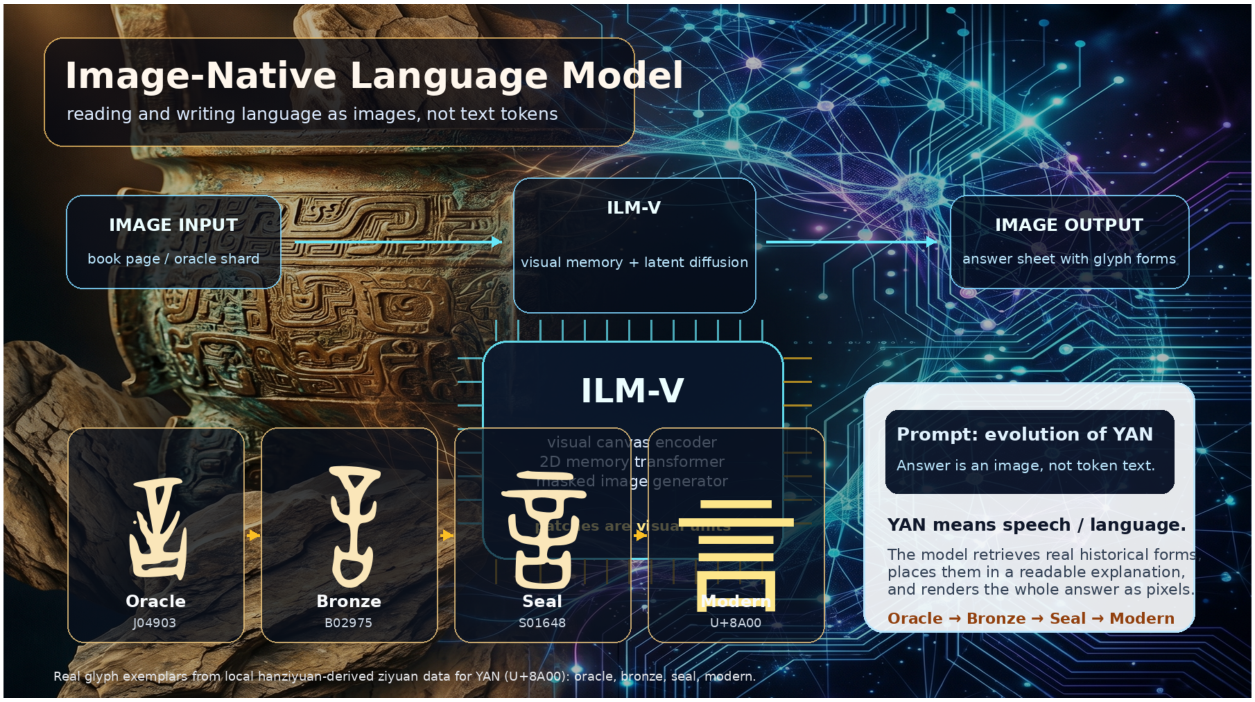

Figure 1: Image-native language modeling concept figure generated with an AgInTi background and overlaid with real local ziyuan glyph exemplars for `YAN` (U+8A00). The answer is a rendered image containing both modern explanation and historical forms.

The missing combination is direct: train a model to read and write language as images, while preserving the visual structure of scripts and historical glyphs.

3 Architecture

Figure 2 gives the proposed system. The main components are:

1. **Visual encoder.** A patch/conv hybrid converts glyph tiles, line strips, and page crops into visual latent grids.
2. **Visual memory transformer.** A 2D Transformer maintains cross-panel state with relative position and sparse/window attention.
3. **Latent generator.** A masked diffusion or MaskGIT-like prior fills an output canvas conditioned on input canvas latents.
4. **Image decoder.** A VAE/VQ-VAE decoder converts latents back to readable output images.
5. **Auxiliary readers and critics.** OCR/readability heads evaluate the output, but should not become the primary hidden path.

The internal units are visual patches or visual codebook entries, not BPE, wordpiece, or Unicode language tokens.

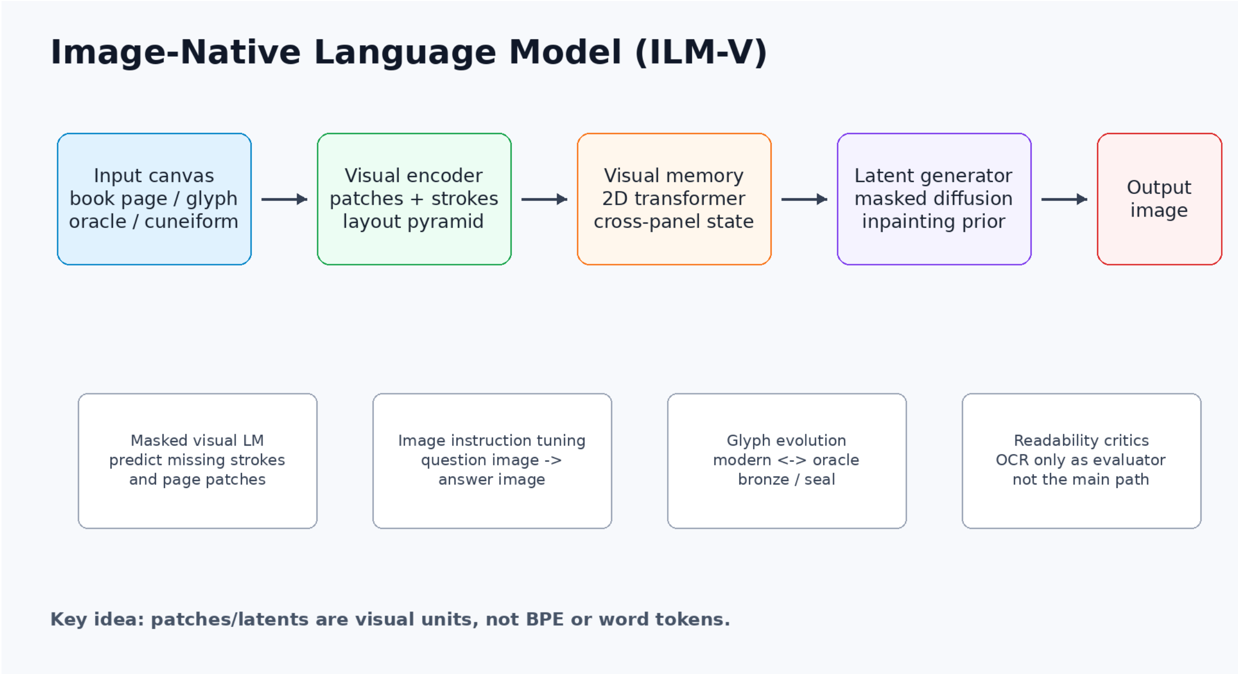

Figure 2: ILM-V architecture. The model consumes input canvases, encodes them into visual latents, maintains a 2D visual memory, and denoises or fills output image latents. OCR-style readers are only auxiliary evaluators, not the main generation path.

4 Data

The project already has a strong historical Chinese glyph source outside the tracked repo. A local snapshot at `/home/lachlan/ProjectsLFS/incoder/data/historic` contains:

Item	Count
Characters	9,055
Glyph records	84,642
Oracle	32,216
Bronze	23,500
Liushutong	21,923
Seal	6,996

This is enough to train a first glyph autoencoder and a character-evolution generator. Modern book/page data can be added by rendering multilingual corpora into images and by ingesting scanned pages where licenses permit.

5 Training Objectives

The first ILM-V model should combine the following losses:

1. **Masked visual language modeling:** mask image patches and predict missing visual latents.
2. **Denoising reconstruction:** recover pages and glyphs from blur, noise, damage, dropout, and partial occlusion.

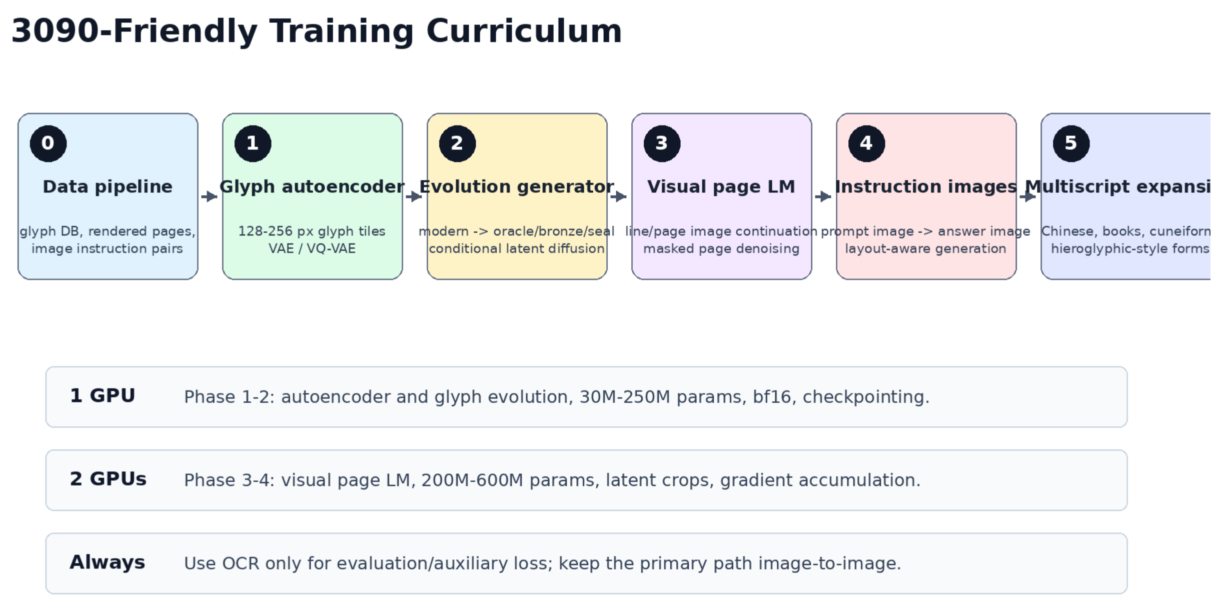

Figure 3: A staged curriculum that fits one or two RTX 3090 GPUs by starting with glyph tiles and latent generation before scaling to page-level image language modeling.

3. **Image-to-image instruction tuning:** render prompts and target answers as images and train conditional generation.
4. **Glyph evolution modeling:** map modern forms to historical stages and historical forms back to modern answer sheets.
5. **Contrastive visual semantics:** align different visual renderings of the same content without making OCR the bottleneck.

6 3090-Friendly Curriculum

Figure 3 shows the practical schedule:

1. Build manifests for glyphs, rendered line images, and rendered instruction pairs.
2. Train a 30M–100M parameter glyph/page autoencoder.
3. Train a 100M–250M parameter conditional glyph evolution generator.
4. Train a 200M–600M parameter visual page language model on latent crops.
5. Add instruction tuning with image prompts and image answers.
6. Expand to additional scripts: cuneiform-like signs, hieroglyphic forms, Arabic, Japanese, Korean, and multilingual book pages.

Mixed precision, gradient checkpointing, latent crops, LoRA/adapters, and gradient accumulation make this path realistic on one or two 24GB GPUs.

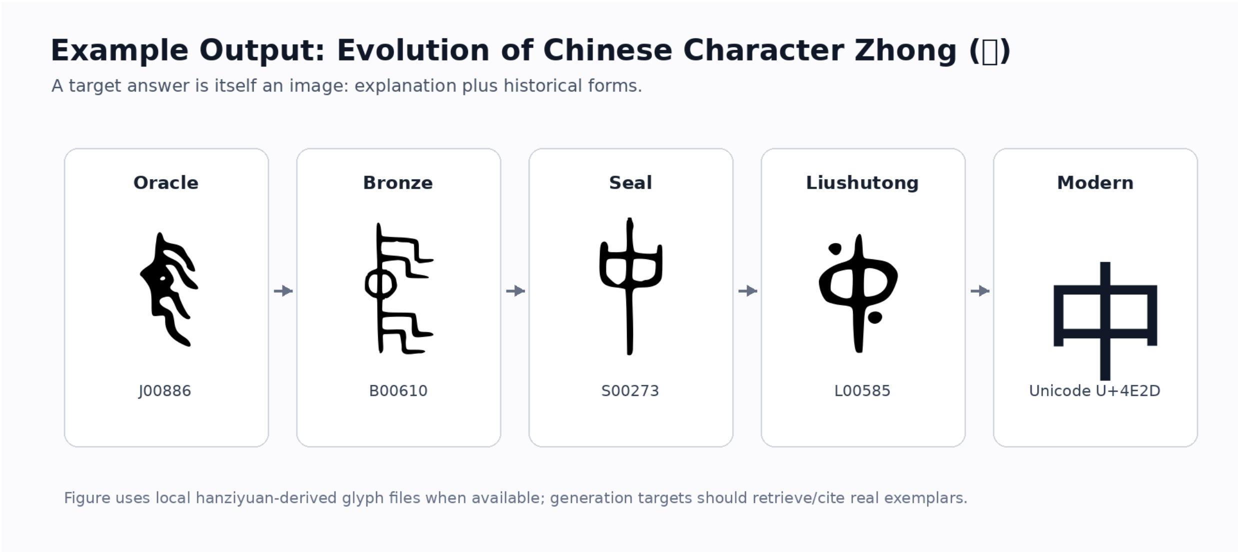

Figure 4: Target output style for a glyph-evolution question. The answer is not merely text: it is a readable image with historical visual forms. The example uses local hanziyuan-derived glyph files when available.

7 Target Demonstration

The first demo should answer a rendered visual prompt: "Explain the evolution of the Chinese character zhong." The target answer image should contain:

- a title and concise modern explanation,
- a historical timeline with real glyph exemplars,
- stage labels and citations,
- modern-language text as rendered image.

This directly tests whether the model can combine language understanding and visual-script generation.

8 AgInTi Artifact Loop

The figure brief for AgInTi-style refinement is stored in the local AgInTi prompts subfolder. The current paper uses scripted PNG generation to keep the artifact reproducible.

9 Evaluation

The model should be evaluated as both a language model and a visual generator:

- OCR readability of generated modern text images, used only as an external metric.
- Human readability and task-success ratings.
- Glyph-stage retrieval accuracy: generated glyphs should retrieve correct characters and stages.

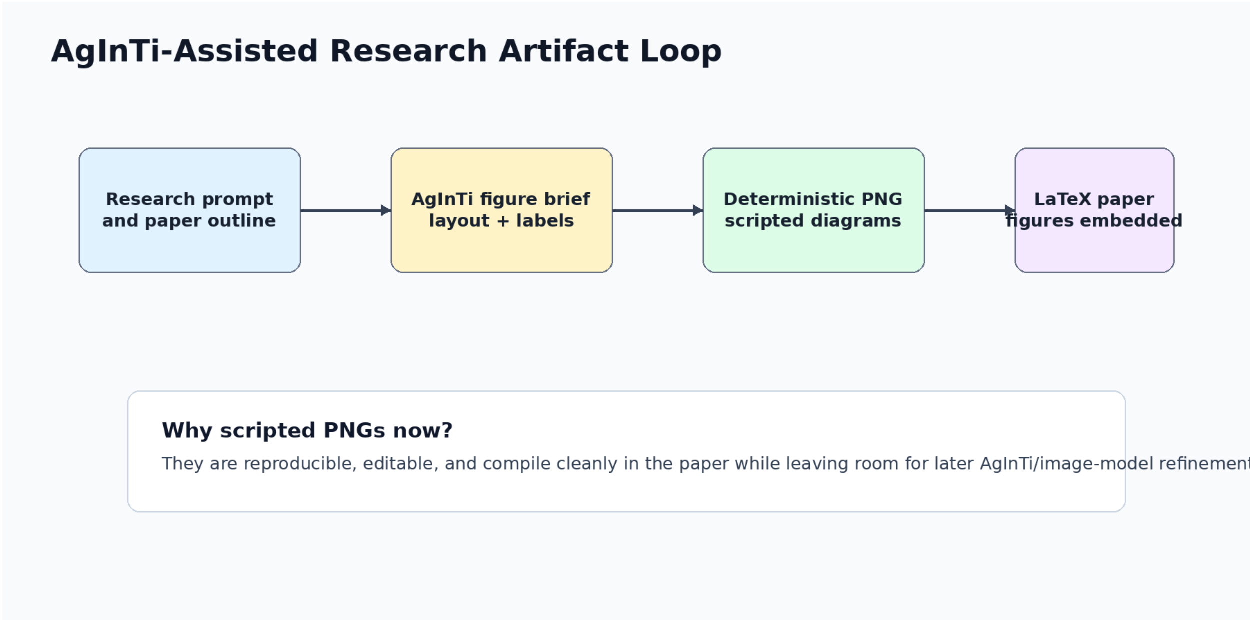

Figure 5: AgInTi-assisted artifact loop. The current figures are deterministic scripted PNGs so the paper is reproducible; the figure brief can later be sent to an interactive AgInTi/image agent for visual refinement.

- Historical plausibility: distance to nearest real glyph exemplars in stage-specific embedding space.
- Layout validity: no text/panel overlap, stable reading order, correct answer composition.
- Image-to-image QA accuracy on rendered prompts and rendered answers.

10 Conclusion

ILM-V is a feasible research direction because the pieces exist: OCR-free visual readers, token-free language models, visual latent autoencoders, masked image generators, and diffusion language models. The contribution is to make visible writing itself the modeling substrate. The local Chinese etymology dataset gives a concrete first domain where image output is essential rather than decorative.

References

[1] Brown et al. Language Models are Few-Shot Learners. `https://arxiv.org/abs/2005.14165`

[2] OpenAI. GPT-4 Technical Report. `https://arxiv.org/abs/2303.08774`

[3] OpenAI. GPT-4o API model documentation. `https://developers.openai.com/api/docs/models/gpt-4o`

[4] Meta AI. The Llama 3 Herd of Models. `https://arxiv.org/abs/2407.21783`

[5] Kim et al. OCR-free Document Understanding Transformer. `https://arxiv.org/abs/2111.15664`

[6] Lee et al. Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding. https://arxiv.org/abs/2210.03347

[7] Clark et al. CANINE: Pre-training an Efficient Tokenization-Free Encoder. https://arxiv.org/abs/2103.06874

[8] Xue et al. ByT5: Towards a Token-Free Future. https://arxiv.org/abs/2105.13626

[9] Yu et al. MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers. https://arxiv.org/abs/2305.07185

[10] Austin et al. Structured Denoising Diffusion Models in Discrete State-Spaces. https://arxiv.org/abs/2107.03006

[11] Li et al. Diffusion-LM Improves Controllable Text Generation. https://arxiv.org/abs/2205.14217

[12] Lou et al. Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution. https://arxiv.org/abs/2310.16834

[13] Nie et al. Large Language Diffusion Models. https://arxiv.org/abs/2502.09992

[14] Chang et al. MaskGIT: Masked Generative Image Transformer. https://arxiv.org/abs/2202.04200

[15] Rombach et al. High-Resolution Image Synthesis with Latent Diffusion Models. https://arxiv.org/abs/2112.10752

[16] Peebles and Xie. Scalable Diffusion Models with Transformers. https://arxiv.org/abs/2212.09748